methods/computer/dynamic_methods.rb

```
class Computer
  def initialize(computer_id, data_source)
    @id = computer_id
    @data_source = data_source
  end

  def self.define_component(name)
    define_method(name) do
      info = @data_source.send "get_#{name}_info", @id
      price = @data_source.send "get_#{name}_price", @id
      result = "#{name.capitalize}: #{info} ($#{price})"
      return "* #{result}" if price >= 100
      result
    end
  end

  define_component :mouse
  define_component :cpu
  define_component :keyboard
end
```

Note that the three calls to define_component are executed inside the definition of Computer, where Computer is the implicit self. Because you're calling define_component on Computer, you have to make it a class method.

You quickly test the slimmed-down Computer class in irb and discover that it still works. It's time to move on to the next step.

Step 3: Sprinkling the Code with Introspection

The latest Computer contains minimal duplication, but you can push it even further and remove the duplication altogether. How? By getting rid of all those calls to define_component. You can do that by introspecting the data_source argument and extracting the names of all components:

methods/computer/more_dynamic_methods.rb

```
class Computer
  def initialize(computer_id, data_source)
    @id = computer_id
    @data_source = data_source
    data_source.methods.grep(/^get_(.*)_info$/) { Computer.define_component $1 }
  end

  def self.define_component(name)
    define_method(name) do
      # ...
    end
  end
end
```